



Scenes System: SceneLoader, DuplicateBag, FadeManager, LevelManager

SceneLoader.

В Unity есть `SceneManager.LoadScene()`, который позволяет загрузить нужную сцену. Однако это простой способ, которого мне недостаточно. Мне нужна надстройка над загрузкой сцены.

Во-первых, стоит сразу сказать, что сцену мы будем загружать аддитивно:

`LoadSceneMode.Additive`, - поверх той, что у нас есть сейчас. Сделано это для того, чтобы во всех случаях, всех уровнях и меню, фоново всегда жила сцена `Boot`, которую было правильнее назвать `Persistent` (постоянная). В ней живут все `singleton` менеджеры, `HUD`, `Settings-Menu`, `Prestart-Screen`, `Death-Screen`.

```
1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5
6 public class SceneLoader : Manager<SceneLoader>
7 {
8     private Scene currentScene;
9     private bool isLoading = false;
10    private Coroutine loadRoutine;
11
12    public void LoadScene(string newSceneName, Action onLoaded = null,
13    {
14        if (isLoading) return;
15        if (loadRoutine != null)
16        {
17            return;
18        }
19
20        isLoading = true;
21
22        loadRoutine = StartCoroutine(LoadSceneRoutine(newSceneName, on
23    }
24
25    public void LoadScene(LevelEntry levelEntry, Action onLoaded = nul
26    {
27        if (isLoading) return;
28        if (loadRoutine != null)
29        {
30            return;
31        }
32
33        isLoading = true;
34
35        loadRoutine = StartCoroutine(LoadSceneRoutine(levelEntry.scene
36    }
37
38    /*
39     * Вызвать загрузку уровня мы можем через LoadScene(), передав туда
40     * либо название string формата, либо тип данных LevelEntry, который такж
41     * хранит название сцены.
42
43     * Начальная версия загрузчика была в разы проще, но позже были
```

```

44 исправлены различные ошибки и расширен функционал.
45 В процессе разработки было замечено, что игровой уровень подгружался д
46 из-за чего вся логика игры сыпалась. Сейчас мы контролируем загрузку ч
47 isLoading флага, а также через прямой контроль корoutines.
48 Две проверки могут быть излишни, но они нужны были в процессе поиска р
49
50 Когда мы начинаем загрузку, то устанавливаем флаг и присваиваем в по
51 К слову, теперь мы передаём callback'и в виде onLoaded & onCompleteFad
52 Первый - для контроля загрузки сцены, а второй - для контроля,
53 когда закончится затемнение и экран полностью окажется скрыт.
54 */
55
56 private IEnumerator LoadSceneRoutine(string sceneName, Action onLo
57 {
58     bool fadeDone = false;
59     FadeManager.FadeOutThen(() => fadeDone = true, 1.5f);
60     yield return new WaitUntil(() => fadeDone);
61
62     // Устанавливаем флаг затемнения - выключен. И вызываем из FadeManager
63     // начало затемнения, куда передаём флаг, который станет true, и время
64     // Сразу же начинаем ожидать через WaitUntil() этого момента.
65
66     if (currentScene.IsValid())
67     {
68         yield return SceneManager.UnloadSceneAsync(currentScene);
69         currentScene = default;
70     }
71
72     // Проверяем текущую сцену на валидность (сцена реально лежит в поле).
73     // Если всё нормально, то запускаем асинхронную выгрузку сцены.
74     // А в текущую сцену ложим default.
75
76     AsyncOperation loadOp =
77         SceneManager.LoadSceneAsync(sceneName, LoadSceneMode.Addit
78     yield return loadOp;
79
80     Scene newScene = SceneManager.GetSceneAt(SceneManager.sceneCou
81     SceneManager.SetActiveScene(newScene);
82     currentScene = newScene;
83
84     // К слову, Boot сцена никогда не попадёт в currentScene поле,
85     // потому что она запускается первой и вызывает через отдельный однора
86     // скрипт-загрузку MainMenu.
87
88     // Мы создаём AsyncOperation и присваиваем в него загрузку новой сцены
89     // после чего ожидаем загрузку. Делаем новую сцену активной,
90     // и указываем её как текущую.
91
92     FadeManager.FadeInThen(
93         () => { onCompleteFade?.Invoke(); },
94         1.5f
95     );
96
97     // Запускаем "растемнение" экрана, передаём калбек onCompleteFade
98     // ещё глубже в менеджер затемнения. Там будет обрабатывать корутина,
99     // которая в конце оповестит подписчиков события.
100
101     onLoaded?.Invoke();
102
103     isLoading = false;
104     loadRoutine = null;
105 }
106
107 // Сразу же вызываем подписчиков onLoaded, выключаем флаг и очищаем ко
108 // Когда растемнение экрана закончится - вызовутся подписчики onComplete
109 // На этом загрузка сцены завершена . . .
110

```

```

111     public void SetCurrentScene(Scene scene)
112     {
113         currentScene = scene;
114     }
115 }

```

```

1 private void LoadStage()
2 {
3     if (State == GameState.Loading) return;
4     State = GameState.Loading;
5
6     AudioManager.Instance.StopMusic();
7
8     SceneLoader.Instance.LoadScene(
9         LevelManager.Instance.GetRandomLevel(),
10        () =>
11        {
12            restartLocked = false;
13
14            DifficultyManager.Instance.SetFloor(GameManager.Instance.Fl
15            WalletManager.Instance.ResetWallet();
16            BaseManager.Instance.ResetBase();
17            HudManager.Instance.Reset();
18            HudManager.Instance.Show();
19            State = GameState.Prestart;
20            EnemySpawnerManager.Instance.StopSpawning();
21            EnemySpawnerManager.Instance.SceneEnabled(5);
22        },
23        () => { HudManager.Instance.ShowPrestartButton(); }
24    );
25 }
26 // GameFlowManager.cs

```

Таким образом вызывается данный загрузчик сцен. Мы передаём название сцены, которое получим из LevelManager, а также передаём лямбдой анонимный метод, который будет содержать нужный нам функционал. В первом случае всё, что произойдёт после загрузки сцены. Во втором случае - мы покажем кнопку старта игры.

Баг дублирования.

Я уже упоминал выше, что столкнулся с багом дублируемой загрузки игровых сцен. Времени было потрачено много.

Во-первых, в GameFlowManager (управляющим циклом игры) пришлось сделать контроль состояний:

```

1 public enum GameState
2 {
3     Menu,
4     Loading,
5     Playing,
6     ArtifactChoice,
7     Defeat,
8     Prestart,
9     EscMenu
10 }
11

```

```

12 private void LoadStage()
13 {
14     if (State == GameState.Loading) return;
15     State = GameState.Loading;
16     . . .
17 }
18
19 public void StartPlaying()
20 {
21     if (State != GameState.Prestart) return;
22     State = GameState.Playing;
23     . . .
24 }
25
26 ...
27 // GameFlowManager.cs

```

Это было сделано для того, чтобы избежать повторного вызова состояний игры. Ожидалось, что это тоже могло влиять на дублирование сцен.

Во-вторых, во многих менеджерах были добавлены флаги проверок, которые делают запрос к флагу State из GFM. Прежний скрипт загрузки сцены насчитывал меньшее количество флагов и короутины вызывались без присваиваний null и контроля переменной.

Решение оказалось достаточно примитивным. Изначально последовательность загрузки выглядела так: загружаем сцену, выгружаем активную, делаем активной загруженную. Выходило так, что сцена загружалась дважды, и даже если одна из них выгружалась - другая обязательно оставалась. Бывало, что сцена загружалась в единственном экземпляре, но в таком случае она гарантированно не успевала выгрузиться.

Решение - сначала выгружаем текущую сцену, оставляя только фоновый Persistent, затем загружаем уровень и делаем его активным. В коде работоспособность этого решения выглядела более странно, оно работает.

Код с дублированием сцен:

```

1 using System;
2 using System.Collections;
3 using UnityEngine;
4 using UnityEngine.SceneManagement;
5
6 public class SceneLoader : Manager<SceneLoader>
7 {
8     private Scene currentScene;
9     private bool isLoading = false;
10    private int loadCounter = 0;
11
12    private Coroutine loadRoutine;
13
14    public void LoadScene(string sceneName, Action onLoaded = null)
15    {
16        if (isLoading) return;
17        if (loadRoutine != null)
18        {

```

```

19         Debug.LogWarning("[SceneLoader] LoadScene ignored - already loa
20         return;
21     }
22
23     isLoading = true;
24
25     Debug.Log($"[SceneLoader] LoadScene calle: {sceneName}, isLoading={
26     loadRoutine = StartCoroutine(LoadSceneRoutine(sceneName, onLoaded))
27     //StartCoroutine(LoadSceneRoutine(sceneName, onLoaded));
28 }
29
30 public void LoadScene(LevelEntry levelEntry, Action onLoaded = null)
31 {
32     if (isLoading) return;
33     if (loadRoutine != null)
34     {
35         Debug.LogWarning("[SceneLoader] LoadScene ignored - already loa
36         return;
37     }
38
39     isLoading = true;
40
41     Debug.Log($"[SceneLoader] LoadScene calle: {levelEntry}, isLoading=
42     loadRoutine = StartCoroutine(LoadSceneRoutine(levelEntry.sceneName,
43     //StartCoroutine(LoadSceneRoutine(levelEntry.sceneName, onLoaded));
44 }
45
46 private IEnumerator LoadSceneRoutine(string sceneName, Action onLoaded
47 {
48     loadCounter++;
49     int myId = loadCounter;
50
51     Debug.Log($"[SceneLoader #{myId}] START coroutine, frame={Time.frame
52
53     if (isLoading)
54     {
55         Debug.LogError($"[SceneLoader #{myId}] 🚨 ENTERED coroutine whi
56     }
57
58
59     bool fadeDone = false;
60     FadeManager.FadeOutThen(() => fadeDone = true, 2f);
61     yield return new WaitUntil(() => fadeDone);
62
63     Debug.Log($"[SceneLoader #{myId}] Fade done");
64
65     AsyncOperation loadOp = SceneManager.LoadSceneAsync(sceneName, Load
66     yield return loadOp;
67
68     Debug.Log($"[SceneLoader #{myId}] Scene loaded");
69
70     Scene newScene = SceneManager.GetSceneByName(sceneName);
71     SceneManager.SetActiveScene(newScene);
72
73     if (currentScene.IsValid())
74     {
75         Debug.Log($"[SceneLoader #{myId}] Unloading {currentScene.name}
76         yield return SceneManager.UnloadSceneAsync(currentScene);
77     }
78
79     currentScene = newScene;
80
81     FadeManager.FadeIn(2f);
82
83     Debug.Log($"[SceneLoader #{myId}] END coroutine");
84     onLoaded?.Invoke();
85     isLoading = false;

```

```

86     loadRoutine = null;
87 }
88
89 public void SetCurrentScene(Scene scene)
90 {
91     currentScene = scene;
92 }
93 }

```

BootSceneLoader.

i Одноразовый скрипт отображения загрузочного экрана и загрузка MainMenu сцены.

```

1  using System;
2  using System.Collections;
3  using UnityEngine;
4  using UnityEngine.SceneManagement;
5  using UnityEngine.UI;
6
7  public class BootSceneLoader : MonoBehaviour
8  {
9      [Header("UI References")]
10     [SerializeField] public Slider progressBar;
11     [SerializeField] public Image background;
12     [SerializeField] private string startScene = "MainMenu";
13
14     [Header("Settings")]
15     [SerializeField] private float minLoadTime = 1.5f;
16     [SerializeField] private float smoothSpeed = 0.03f;
17
18     [Header("ClearObjects")] [SerializeField]
19     public GameObject[] clearObjects;
20
21     void Start()
22     {
23         StartCoroutine(LoadMainScene());
24     }
25
26     // Начинаем асинхронную загрузку стартовой сцены, в нашем случае - Main.
27     // Пока загрузка идёт вычисляем положение слайдера.
28
29     private IEnumerator LoadMainScene()
30     {
31         AsyncOperation op = SceneManager.LoadSceneAsync(startScene, LoadMode.NoReleaseObject);
32         op.allowSceneActivation = false;
33
34         float timer = 0f;
35         float targetProgress = 0f;
36         float displayedProgress = 0f;
37
38         /*
39          sceneProgress содержит текущий прогресс загрузки сцены.
40          Операция загрузки возвращает 0.0 - 0.9 (90%) - как результат считывания
41          с диска и загрузки в оперативную память. Вот только мы так и не получим
42          в виде 1, пока не активируем сцену вручную.
43          А автоматическую активацию мы выключаем - op.allowSceneActivation = false
44          Это сделано на случай, если сцена загрузится слишком быстро,
45          а у нас есть ещё один прогресс. Когда мы поделим 0.9\0.9, то получим 1.
46          */
47         while (!op.isDone)
48         {

```

```

49         timer += Time.deltaTime;
50
51         float sceneProgress = Mathf.Clamp01(op.progress / 0.9f);
52         float timerProgress = Mathf.Clamp01(timer / minLoadTime);
53
54         // Выбираем то, что дальше (сгрузим минимум minLoadTime)
55         targetProgress = Mathf.Max(sceneProgress, timerProgress);
56
57         // Плавно приближаем отображаемое значение
58         displayedProgress = Mathf.Lerp(displayedProgress, targetPro
59         progressBar.value = displayedProgress;
60
61
62         // timerProgress через деление time \ minLoadTime переводит секунды в п
63         // загрузки:
64         // 1\1.5 = 0.6        1.5\1.5 = 1        2\1.5 = 1.3
65
66         // Проверка на завершение
67         if (sceneProgress >= 1f && timerProgress >= 1f)
68         {
69             yield return new WaitForSeconds(1f);
70             op.allowSceneActivation = true;
71             while (!op.isDone)
72                 yield return null;
73         }
74
75         // Слайдер работает с процентами в виде 0-1. Поэтому 1.3 = выходу за гр
76         // С помощью Mathf.Clamp01 мы гарантировано будем возвращать не более 1
77         // Я сделал искусственную загрузку длиной в 1.5 секунды + 1 секунда,
78         // когда слайдер дойдёт до 100%.
79
80         yield return null;
81     }
82     SceneManager.SetActiveScene(SceneManager.GetSceneByName(startSc
83     foreach (var clearObject in clearObjects) { Destroy(clearObject
84
85     Scene mainMenuScene = SceneManager.GetSceneByName(startScene);
86     SceneLoader.Instance.SetCurrentScene(mainMenuScene);
87     AudioManager.Instance.PlayMusicType(MusicType.Menu);
88 }
89 }

```

Также укажу, что мы используем здесь `Mathf.Lerp` - известный и простой способ создания плавности. Формула: $A + (B - A) \times t$.

Берём точку старта A, точку куда нужно дойти B, и t - процент расстояния от A и B, который нужно пройти за один текущий кадр.

$t = 0.3 \times 0.016 \times 60 \sim 0.288$ (28.8% от оставшегося)

Допустим начало: $0 + (0.66 - 0) \times 0.3 = 0.198$ (20%).

Следующий кадр: $0.19 + (0.66 - 0.19) \times 0.3 = 0.33$ (в этом примере B ещё не успел поменяться)

Это лишь пример, потому что цикл `while` идёт куда чаще (в примере 0.66 означает target за секунду). На деле мы будем прибавлять по крошечным процентам каждую долю секунды, из-за чего получим плавную загрузку, которая местами всё же может быть быстрой.

Если пойти по примерам дальше, то мы увидим, что каждый следующий прыжок значения всё меньше, чем предыдущие. Это называется асимптотическим приближением.

P.s. расписал это больше для себя и для своего углублённого закрепления, чем для самой документации, но почему нет 😊

FadeManager.

❗ Сцены быстро перещёлкиваются, словно резанные кадры. Иногда это нормально. Например открытию и закрытию меню не всегда нужны какие-то переходы. Но как человек, который много занимался монтажом видео - я люблю когда есть какие-то логичные красивые переходы. Для игры я решил сделать самое простое решение - затемнение экрана и его растемнение.

Напомню, что в момент загрузки уровня я сначала выгружаю одну сцену, а потом загружаю другую. В этот момент камеры в игре нет, а значит у нас чёрный экран. Даже если сделать камеру общей и поместить на Boot (персистент) сцену, то экран всё равно будет чёрным. Скорее всего это будет похоже как если бы просто моргнули. Возможно с легким замиранием.

Чтобы это выглядело чуть мягче и логичнее - Fade подойдёт в самый раз.

```
1 using System.Collections;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 public class FadeManager : Manager<FadeManager>
6 {
7     [Header("References")]
8     [SerializeField] public CanvasGroup fadeCanvas;
9     [SerializeField] public Image fadeImage;
10
11     [Header("Default Settings")] [SerializeField]
12     private float defaultDuration = 1.5f;
13     [SerializeField] private Color defaultColor = Color.black;
14
15     private Coroutine currentFade;
16
17     protected override void Awake()
18     {
19         base.Awake();
20
21         if (fadeCanvas != null)
22         {
23             fadeCanvas.alpha = 0f;
24             fadeCanvas.blocksRaycasts = false;
25         }
26     }
27
28     public static void FadeIn(float duration = -1f, Color? color = null)
29     {
30         if (Instance != null)
31         {
32             Instance.StartFade(false, duration, color);
```

```

33     }
34 }
35
36 public static void FadeInThen(System.Action onComplete, float dura
37 {
38     if (Instance != null)
39     {
40         Instance.StartFade(false, duration, color, onComplete);
41     }
42 }
43
44 // FadeIn \ FadeInThen - прокидывают false в основной метод.
45
46 public static void FadeOut(float duration = -1f, Color? color = nu
47 {
48     if (Instance != null)
49     {
50         Instance.StartFade(true, duration, color);
51     }
52 }
53
54 public static void FadeOutThen(System.Action onComplete, float dur
55 {
56     if (Instance != null)
57         Instance.StartFade(true, duration, color, onComplete);
58 }
59
60 // FadeOut \ FadeOutThen - прокидывают true.
61
62 // Then прописка показывает о том, что можно дополнительно прокинуть т
63 // калбек (порцию методов, которые вызовутся когда событие сработает).
64
65 private void StartFade(bool fadeOut, float duration, Color? color
66 {
67     if (currentFade != null)
68     {
69         StopCoroutine(currentFade);
70     }
71
72     currentFade = StartCoroutine(FadeRoutine(fadeOut, duration, co
73 }
74
75 /*
76  FadeRoutine управляет процессом. Если мы получили false,
77 то значит сейчас у нас включен чёрный квадрат, и нам пора постепенно п
78 его alpha (прозрачность). Если прокидываем true - значит прозрачность
79 а квадрата невидно и нам надо плавно его включить (затухание).
80 Мы передаём в Lepr внутри While параметры - текущую прозрачность и ожи
81 Благодаря множеству итераций и самому Lepr делаем плавное затухание
82 "растухание".
83 */
84 private IEnumerator FadeRoutine(bool fadeOut, float duration, Colo
85 {
86     if (fadeCanvas == null)
87     {
88         yield break;
89     }
90
91     fadeCanvas.blocksRaycasts = true;
92     fadeImage.color = color ?? defaultColor;
93
94     float time = 0f;
95     float start = fadeCanvas.alpha;
96     float end = fadeOut ? 1f : 0f;
97
98     if (duration <= 0)
99     {

```

```

100         duration = defaultDuration;
101     }
102
103     while (time < duration)
104     {
105         time += Time.deltaTime;
106         fadeCanvas.alpha = Mathf.Lerp(start, end, time / duration)
107         yield return null;
108     }
109
110     fadeCanvas.alpha = end;
111     if (!fadeOut)
112     {
113         fadeCanvas.blocksRaycasts = false;
114         yield return new WaitForSeconds(1f);
115     }
116
117     onComplete?.Invoke();
118     currentFade = null;
119 }
120 }
121

```

LevelManager.

i В демо версии планируется несколько малоотличающихся уровней в одном стиле. Так как это бесконечный tower defence, то уровень - это просто этаж. Здесь нет градации сложности как в сюжетных tower defence, где каждый раз появляются новые уровни, новые механики и новые враги. Поэтому (по крайней мере на этапе данного демо) достаточно будет просто выдавать случайный уровень из всех имеющихся.

Чтобы уменьшить повторяемость понадобится хранить информацию о уровнях, что уже выпали до этого.

Я храню список всех возможных уровней. И в случае если их больше пяти (что и планируется), то мы будем хранить очередь из трёх уровней. В момент генерации уровня под следующий этаж просто будем проверять на совпадения с этой очередью. Если нету, то выпавший попадает в очередь, выпихивая вошедшего ранее.

```

1  using System.Collections.Generic;
2  using NUnit.Framework;
3  using UnityEngine;
4
5  public class LevelManager : Manager<LevelManager>
6  {
7      [Header("Levels")]
8      [SerializeField] private List<LevelEntry> levels = new();
9
10     private int currentLevelIndex = -1;
11     private Queue<int> lastPlayed = new(3);
12
13     public int CurrentLevelIndex => currentLevelIndex;
14     public LevelEntry CurrentLevel =>
15         currentLevelIndex >= 0 ? levels[currentLevelIndex] : null;
16

```

```
17 public LevelEntry GetRandomLevel()
18 {
19     if (levels.Count == 0)
20     {
21         Debug.LogError("No levels configured!");
22         return null;
23     }
24
25     if (levels.Count < 5)
26     {
27         currentLevelIndex = Random.Range(0, levels.Count);
28         return levels[currentLevelIndex];
29     }
30
31     int index;
32     int safety = 0;
33
34     do
35     {
36         index = Random.Range(0, levels.Count);
37         safety++;
38     }
39     while (lastPlayed.Contains(index) && safety < 20);
40
41     RegisterPlayed(index);
42     currentLevelIndex = index;
43     return levels[index];
44 }
45
46 private void RegisterPlayed(int index)
47 {
48     lastPlayed.Enqueue(index);
49
50     if (lastPlayed.Count > 2)
51         lastPlayed.Dequeue();
52 }
53 }
```